Estructura SWITCH para manejo de toma de decisiones:

```
switch (variable){
case valor1:
        sentencias;
        break;
case valor2:
        sentencias;
        break;
        .....
default:
    sentencias;
            }
```

Para operador SWITCH la variable principal (variable) debe ser o tipo entero o character (int o char). Float y double no se permiten!

```
#include <iostream.h>
#include <conio.h>
#include <stdio.h>
#include <math.h>
#include <stdlib.h>

void main()
{

int nota, dec;

printf("nota=");
scanf("%d", &nota);
if (0<= nota && nota <=50){
            dec=nota/10;
            switch (dec){
```

```
            case 5: case 4:
            printf(" %2d Aprueba.  Muy bien. \n", nota);
            break;
            case 3:
            printf(" %2d Aprueba. \n", nota);
            break;
            case 2:
            printf(" %2d Puede habilitar. \n", nota);
            break;
            default:
            printf(" %2d Reprueba. \n", nota);

                    }
}
else
printf(" %d Nota inadecuada. \n", nota);
getch();
}
```

Errores en uso de operador SWITCH:

- **Olvidar poner una instrucción *break* cuando es necesaria.**
- **Omitir el espacio entre la palabra case y el valor integrado que se evalúa dentro de una estructura *switch*, puede ocasionar un error lógico. Por ejemplo, escribir *case3:* en lugar de *case 3:***

Recomendaciones:

- **En una estructura *switch* que lista la clausula *default* al ultimo, la clausula *default* no requiere una instrucción *break*.**
- **Las clausulas *case* y la clausula *default* pueden ir en cualquier orden, pero es recomendable poner *default* al último.**

Ejercicio: en el programa anterior probar poner default en segundo lugar; ponerlo con break y sin break, observar los resultados.

Ejemplo: Calculo de sueldo.

```
#include <stdio.h>
#include <conio.h>

int main()
{
      int nivel;
      float salario, slanuevo;
      printf("Introduce el nivel de antiguedad del trabajador:
");
      scanf("%d",&nivel);
      printf("Introduce tu salario: ");
      scanf("%f",&salario);
      switch (nivel) {
        case 5:
                slanuevo=salario+(salario*.035);
                printf("\nTu salario es:%f\n",slanuevo);
                break;
        case 6:
                slanuevo=salario+(salario*.041);
                printf("\nTu salario es: %f\n",slanuevo);
                break;
        case 7:
                slanuevo=salario+(salario*.048);
                printf("\nTu salario es: %f\n",slanuevo);
                break;
        case 8:
                slanuevo=salario+(salario*.053);
                printf("\nTu salario es: %f\n",slanuevo);
                break;
        default:
                printf("\nTu salario es: %f\n",salario);
}
getch();
return 0;
}
```

CONTINUE

Permite saltar de la parte restante del cuerpo de un bucle for, while o do/while y pasar a empezar una nueva iteración del ciclo.

EJEMPLO: Calcular la suma de los enteros no múltiplos de 10, entre n1 inclusive y n2 inclusive.

```
#include <iostream.h>
#include <conio.h>
#include <stdio.h>
#include <math.h>
#include <stdlib.h>

void main()
{

int n1, n2, div, res, i;
double sx;


div=10;
printf("n1 n2 : ");
scanf("%d%d", &n1, &n2);
sx=0.0;
 for (i=n1; i<=n2; i++)
 {
   res =i%div;
   if(res ==0) continue;
     sx+=i;
printf("%5d\n", i);
 }
printf("suma x= %lf", sx);
```

```
getch();
}
```

Ejercicio: que hace siguiente código:

```
y = 0;
x = 0;
while(x < 1000)
{
   x++;
   if(y >= 100) continue;
   y++;
}
```

Ejercicio clase: cambiar el código de tal forma que calcula la suma de cuadrados entre n1 y n2 menos cuando se trata de números múltiplos de 10 o de 7.

Estructura break:
Se usa en los operadores for, while, do while, if, switch para interrumpir el ciclo o bucle y saltar fuera de ciclo para el operador siguiente después de }. Provoca la salida inmediata de estructura donde se aplica.

Ver programa ejemplo_break

ESTRUCTURA GOTO

- Transferencia de control a uno de los muchos posibles destinos dentro de un programa (a una instrucción con etiqueta indicada).
- Una etiqueta es un identificador seguido de dos puntos.

- Una etiqueta debe aparecer en la misma función que la instrucción *goto* que hace referencia a ella.

- Salida de varios bucles mediante una solo instrucción.

- “Programación estructurada” = “Eliminación de *goto*”

- Esta demostrado que programas se puede escribir sin goto. (Bohm C., Jacopini G. “Flor Diagams, Turing Machines, and Languages with Only Two Formation Rules”).

- La instrucción *goto* debe utilizarse solo en aplicaciones orientadas al rendimiento. La instrucción *goto* carece de estructura y puede ocasionar que los programas sean más difíciles de depurar, mantener y modificar.

```
{
 …..
 goto final1;
 ….
final1: …    // “final1” es la etiqueta
 ....
}
```

Ejemplo de cambiar uso de goto por programación estructurada:

```
x = 0;

Bucle:  x++;

if (x < 1000) goto Bucle;
```

Este ejemplo hace exactamente que el operador estructurado *for:*

```
for (x = 0; x < 1000; x++);
```

Ejercicio: Escriba un programa que utilice instrucciones goto para simular una estructura de ciclos para imprimir un cuadro de asteriscos, como se muestra en dibujo. El programa debe utilizar solamente las siguientes instrucciones de salida:

cout << '*';

cout << ' ';

cout << endl;

Puede usar *if* pero no puede usar ningún operador de ciclo.

*****

*** ***

*** ***

*** ***

*****

Ejercicios:

1. Calcular la suma de los primeros 10 números de Fibonacci (averiguar en Internet las formulas) o sea desde F(0) hasta F(10) pero excepto F(4). Utilizar operador "continue" para que en su suma no aparezca F(4).
2. Escribir un programa que dado un entero positivo n, calcule a lista primos menores o iguales a n (para cada

numero deben averiguar si el es divisible por los enteros entre 2 y la raíz cuadrada de n) y al final imprime la cantidad de ellos.